

AI-Native Self-Healing O-RAN Network using a Digital Twin

Open Fronthaul Synchronisation-Plane (S-Plane) Timing Security

Technical Report and Foundational Briefing

Prepared for the project team — no prior telecom background assumed

Field	Value
Project	AI-Native Self-Healing O-RAN Network using a Digital Twin
Scope	Open Fronthaul S-plane (timing) fault-vs-attack discrimination and governed recovery
Status	Software complete and validated; Tier-3 hardware validation not started
Repository state	Commit 4e4bd16, 56 automated tests passing, synchronised to origin
Classification	Reproducible research prototype — not production certification
Report date	7 August 2026

How to read this report. Part I assumes no knowledge of mobile networks or time synchronisation and builds the necessary background from first principles. Parts II–IV describe the problem, the system and the experimental method. Part V narrates what was actually done, including the approaches that failed. Parts VI–VII present results and the limits of what software alone can achieve. Part VIII documents how the work was produced, including the use of AI coding agents and the verification regime adopted in response. A glossary and a complete repository map appear at the end.

Contents

Part I	Foundations: from zero	3
1.1	Why a mobile network needs a shared clock	3
1.2	What O-RAN is, and why it matters	3
1.3	The fronthaul and its four planes	4
1.4	How time is actually distributed: PTP, SyncE, GNSS	4
1.5	The two numbers that define this project	5
Part II	The problem	6
2.1	The threat landscape	6
2.2	Fault versus attack: why they look identical	6
2.3	Where the state of the art stops	7
2.4	Research question	7
Part III	System architecture	8
3.1	Pipeline overview	8
3.2	Component walkthrough	8
3.3	Why these machine-learning methods	9
3.4	The 28 features	10
Part IV	Methodology	11
4.1	Three tiers of validation	11
4.2	Data provenance — stated bluntly	11
4.3	Designing an evaluation that cannot cheat	11
4.4	How to read the metrics	12
Part V	What we actually did	13
Part VI	Results	15
Part VII	Negative results and the fundamental limit	18
Part VIII	How this work was produced	20
Part IX	What remains: the Tier-3 roadmap	22
Part X	Glossary	24
Appendix A	Repository map, file by file	26
Appendix B	Reproduction instructions	29

Part I — Foundations: from zero

This part assumes you have never worked with mobile networks or time synchronisation. Everything the rest of the report relies on is built here. If you already know what PTP and the O-RAN fronthaul are, skip to Part II.

1.1 Why a mobile network needs a shared clock

A 5G base station does not transmit continuously. It transmits in precisely scheduled slots, and neighbouring cells must not talk over one another. Several core techniques depend on every participating radio agreeing on *what time it is* to a startling degree of precision:

- **Time-division duplex (TDD).** Uplink and downlink share one frequency, separated only in time. If two nearby cells disagree about when the downlink slot ends, one transmits while the other is listening — and deafens it.
- **Carrier aggregation and MIMO.** Combining signals from multiple antennas requires their relative phase to be known, which is a timing statement.
- **Handover.** Passing a moving user between cells requires both cells to share a timebase.

The consequence is blunt: **time is not a convenience in a radio network, it is part of the physical layer.** If timing degrades, the network does not slow down gracefully — cells become mutually interfering and service collapses. This is why timing is a security concern and not merely an operational one.

An analogy that holds up

Think of an orchestra where musicians are in separate rooms and can only hear a conductor's metronome relayed to each of them. If one musician's metronome drifts by a fraction of a beat, they play over their neighbour and the performance degrades for everyone — not just for them. Now suppose an adversary can subtly alter one musician's metronome. That is the attack surface this project addresses.

1.2 What O-RAN is, and why it matters

Traditionally a mobile operator bought a base station as a sealed unit from a single vendor: radio, processing and software together, with proprietary internal interfaces. **O-RAN** (Open Radio Access Network) is an industry initiative to split that unit into standardised, interoperable components so operators can mix vendors.

The base station is decomposed into three pieces:

Component	Full name	Role
O-RU	O-RAN Radio Unit	The radio itself — antennas, RF front end, low-level physical layer. Sits at the mast.
O-DU	O-RAN Distributed Unit	Real-time baseband processing. Usually in a nearby cabinet or edge site.
O-CU	O-RAN Central Unit	Higher-layer, less time-critical processing. Can be far away, often virtualised.

Splitting the box open creates the opportunity O-RAN exists for — and simultaneously creates a new problem. Interfaces that were once internal, proprietary and physically inaccessible are now standardised links running over ordinary Ethernet. **A standardised, documented interface is also a documented attack surface.** The O-RAN Alliance's own security working group (WG11) maintains a threat model precisely because of this.

1.3 The fronthaul and its four planes

The link between the O-DU and the O-RU is called the **Open Fronthaul**. It is the most timing-critical link in the system, because it carries the raw radio samples that must be emitted from the antenna at exactly the right instant.

Traffic on the fronthaul is organised into four logical **planes**. A plane is a category of traffic, not a physical wire — all four share the same cable:

Plane	Name	Carries
C-plane	Control	Scheduling commands: which resources to use in which slot.
U-plane	User	The actual radio sample data (the payload).
S-plane	Synchronisation	Timing. Keeps the O-RU's clock aligned with the network. This project's entire subject.
M-plane	Management	Configuration, monitoring, software updates, status reporting.

A frequent point of confusion worth settling now: **planes and OSI layers are two different maps**. Planes classify traffic by *function* (what it is for). OSI layers classify by *how* data moves (physical, data link, network, transport...). The S-plane is a functional plane; in OSI terms its traffic sits at Layer 1 (SyncE, on the wire) and Layer 2 (PTP carried directly in Ethernet frames). Saying 'the S-plane is Layer 2' is loose; saying 'S-plane traffic is carried at L1 and L2 in the O-RAN fronthaul profile' is correct.

1.4 How time is actually distributed

Three mechanisms work together. Understanding their division of labour is essential, because the attacks and the defences differ for each.

PTP — Precision Time Protocol (IEEE 1588)

PTP distributes **phase and time-of-day**: the actual instant, not just the tick rate. One device is elected the **grandmaster** (the authoritative clock, usually GPS-disciplined). Others are slaves that continuously correct themselves toward it.

The mechanism is a timed message exchange. Four timestamps are collected:

Symbol	Event	Measured by
t1	Master sends a Sync message	Master
t2	Slave receives that Sync	Slave
t3	Slave sends a Delay_Req message	Slave
t4	Master receives that Delay_Req	Master

From these four numbers the slave computes two quantities:

```
meanPathDelay = ((t2 - t1) + (t4 - t3)) / 2
offsetFromMaster = (t2 - t1) - meanPathDelay
```

Worked example. Suppose the true one-way network delay is 50,000 ns and the slave's clock is 200 ns ahead of the master. Then $t2 - t1 = 50,000 + 200 = 50,200$ ns (the message took 50 us but the slave timestamps it with a clock reading 200 ns high). Going the other way, $t4 - t3 = 50,000 - 200 = 49,800$ ns. The mean is 50,000 ns — the true delay, recovered. The offset is $50,200 - 50,000 = 200$ ns — the true error, recovered. The slave then steers its clock by -200 ns.

The critical assumption. That arithmetic only works if the path delay is the same in both directions. PTP assumes symmetry. An attacker who can delay traffic in one direction only — without forging a single message — injects an undetectable timing error. This is why timing security is hard: the protocol's core mathematics rests on an assumption an adversary can violate silently.

BMCA — how the grandmaster is chosen

Devices advertise their clock quality in **Announce** messages, carrying fields such as priority1, clockClass, clockAccuracy and stepsRemoved. The Best Master Clock Algorithm compares these and elects the best available source. This is automatic and unauthenticated in the base standard — an attacker who simply *claims* to be an excellent clock can win the election and become the time source for everything downstream. That is attack family 1 in this project.

SyncE — Synchronous Ethernet (ITU-T G.8261/G.8264)

SyncE distributes **frequency** only — the tick rate, not the time of day — by recovering a clock from the physical Ethernet signal itself. It is more robust than PTP because it lives in the physical layer, but it cannot tell you what time it is. In practice SyncE holds frequency steady while PTP supplies phase. Quality is advertised through ESMC messages carrying a **Quality Level (QL)**.

GNSS — satellite time

The grandmaster typically derives its authority from GPS or another GNSS constellation. This is the root of trust for the entire timing tree — and it is a faint radio signal from orbit that can be jammed or spoofed with inexpensive equipment. If GNSS lies, PTP faithfully distributes the lie. This is attack family 5, and Part VII explains why it proved to be the hardest.

Holdover — what happens when the reference is lost

If a device loses its reference it does not immediately fail. It enters **holdover**: it free-runs on its own internal oscillator, drifting slowly and predictably according to that oscillator's specification. Holdover is a normal, benign condition. It is also, critically, *what a GNSS attack is designed to look like*.

1.5 The two numbers that define this project

Quantity	Value	Meaning
Time-error budget	~100 ns	Maximum tolerable deviation from correct time. Beyond this, radio performance degrades and cells interfere.
Failure window	~2 s	Published time from the onset of a timing attack to base-station service failure. Any defence must detect and act inside this.

100 nanoseconds is one ten-millionth of a second — roughly the time light takes to travel 30 metres. Two seconds is the entire budget for noticing a problem, deciding what it is, choosing a response and applying it. Those two numbers together are why this cannot be a human-in-the-loop process, and why it must be automated.

Part II — The problem

2.1 The threat landscape

Working from the O-RAN Alliance WG11 threat model and the peer-reviewed literature, five families of attack on fronthaul timing are relevant. This taxonomy was compiled as a distinct research task and drove the entire experimental programme.

#	Family	Mechanism	Status in this project
1	Spoofing / grandmaster impersonation	Forged Announce messages claiming superior clock quality win the BMCA election; downstream clocks re-parent to the attacker.	Covered (H1 <i>spoof</i>)
2	Replay	Legitimate Sync/Follow_Up messages are captured and retransmitted out of context, corrupting the offset computation without forging anything.	Covered (H1 <i>replay</i>)
3	Delay / MITM asymmetry	Asymmetric delay injected on one direction only. Nothing is forged; the attack exploits PTP's symmetry assumption directly.	Open — needs redundant paths
4	Denial of service / flooding	Excessive PTP traffic exhausts the clock's ability to serve legitimate timing.	Covered (H1 <i>dos</i>)
5	Time-source manipulation	GNSS spoofing or jamming corrupts the reference the grandmaster trusts. PTP then faithfully distributes a wrong but internally consistent time.	Covered (jam); spoof unresolved

2.2 Fault versus attack: why they look identical

This is the crux of the project, and it deserves care.

When a timing anomaly occurs, the immediately observable symptom is nearly always the same: **the clock offset starts to drift**. That single symptom can arise from entirely benign causes or entirely malicious ones:

Benign fault (H0)	Malicious attack (H1)
GNSS reception lost; grandmaster enters holdover and drifts on its oscillator.	GNSS spoofed; grandmaster believes it is healthy but distributes wrong time.
Network congestion increases packet delay variation, adding noise to offset estimates.	Attacker injects asymmetric delay, biasing offset estimates.
SyncE quality degrades after an upstream link failure.	Attacker forges superior clock attributes and captures the BMCA election.
Legitimate planned grandmaster failover during maintenance.	Rogue master impersonation.

The two columns require **opposite responses**. For a benign holdover, the correct action is usually to wait or ride holdover — the reference will return. For an attack, the correct action is to isolate the rogue source and fail over. Applying the wrong response makes the outage worse: isolating a source that was merely in holdover destroys a working path, while riding holdover during an active spoof hands the attacker more time.

This is the project's central claim. A system that only reports 'something is wrong' leaves the operator with a coin flip under a two-second deadline. Distinguishing H0 from H1 is what makes an automated response safe enough to attempt at all.

2.3 Where the state of the art stops

The strongest published work in this specific area is **TIMESAFE** (ACM Transactions on Privacy and Security, 2025), from Northeastern University. It demonstrates passive detection of PTP attacks on a real 5G testbed at approximately 97.5% accuracy, and releases its capture data publicly — data this project uses.

TIMESAFE stops at detection, and says so explicitly, noting that the O-RAN specification does not define what actions to take upon detecting a threat. It also does not separate a malicious attack from a benign timing fault. Those two gaps — **discrimination** and **response** — are precisely the space this project occupies.

2.4 Research question

Given only synchronisation telemetry from an O-RAN fronthaul, can a system (a) distinguish a benign timing fault from a malicious timing attack, (b) select a recovery action that is verified to keep time error inside budget, and (c) commit that action within the ~2 second failure window — while behaving safely when it encounters a situation it has never seen?

Note the final clause. It was added part-way through the project, in response to experimental evidence, and it turned out to matter more than anything else. Part V explains why.

Part III — System architecture

3.1 Pipeline overview

The system is a sequential pipeline. Telemetry enters at the left; a governed action leaves at the right. Every stage can veto and route to a conservative default.

Stage	Function
1. TELEMETRY	Raw PTP/SyncE/GNSS observations from a simulator, a packet capture, or a live linuxptp daemon. Normalised to a single canonical schema with explicit validity flags.
2. WINDOWING	Samples grouped into 0.4 s windows stepped every 0.2 s. Each window reduced to 28 numeric features.
3. VALIDITY GATE	Fail-closed check. If the window cannot prove it was built from observed telemetry, it is routed to UNKNOWN immediately — before any model runs.
4. ANOMALY DETECT	Is anything wrong at all? Threshold on time error and delay variation.
5. NOVELTY (OPEN-SET)	Does this resemble anything seen in training? Four Isolation Forests, one per feature group. If not recognised, label UNKNOWN.
6. CLASSIFY H0/H1	Random Forest decides benign fault versus attack, for recognised events only.
7. PERSISTENCE	Require 2 of the last 3 windows to agree before acting. Suppresses single-window noise.
8. DIGITAL TWIN	Forecast the resulting time error for each candidate recovery action, with a fidelity (trust) score.
9. GOVERNED ACTION	Commit the best action only if the twin says it beats doing nothing and the twin is trustworthy. Otherwise safe default.

Available recovery actions are: switch timing source (LLS-C1/C2/C3), fail over to GNSS, enter holdover, isolate a rogue master, reroute the path, or apply the conservative safe default.

3.2 Component walkthrough

Ingestion and the canonical schema

Plain: different sources of timing data are translated into one common format, so the rest of the system does not care where the data came from.

Technical: ingest/schema.py defines a canonical telemetry schema. Three adapters populate it: pcap_ingest.py (parses PTP-over-Ethernet captures and recomputes offset via the IEEE-1588 four-timestamp method), linuxptp_ingest.py (parses ptp4l/phc2sys servo logs), and live_collect.py (polls a running ptp4l daemon through the pmc management client). Because all three emit the identical schema, the unchanged feature extractor, classifier, twin and healing loop operate on simulated, recorded and live data without modification. This design decision is what made later real-data validation possible at all.

Feature extraction

Plain: a window of raw measurements is boiled down to 28 summary numbers that describe its character.

Technical: telemetry/features.py computes windowed statistics across six logical groups (Section 3.4). Crucially, windows also carry **provenance** — telemetry_valid and valid_sample_fraction — recording whether every underlying sample was genuinely observed. This is what makes the fail-closed gate possible.

The fail-closed validity gate

Plain: if the system did not actually receive data, it must not say 'everything is fine'.

Technical: healing/loop.py::telemetry_is_valid runs at the top of choose_action, before anomaly detection. It rejects windows that declare themselves invalid, have an incomplete valid-sample fraction, contain NaN or non-finite features, or carry no provenance metadata at all. Rejected windows route to UNKNOWN with a conservative safe default, on a dedicated persistence channel so outages do not contaminate the attack-detection history. Part V.6 explains the defect that made this necessary.

Open-set novelty detection

Plain: a second opinion that asks 'have I ever seen anything like this before?' rather than 'which of my known categories is this?'

Technical: discriminator/openset.py fits one Isolation Forest per feature group (timing, protocol regularity, message rate, BMCA). A window is flagged novel if **any** group flags it. Per-group thresholds are calibrated with a weighted Sidak correction so the combined false-alarm rate on known data meets a configured budget (2%). Group-wise operation matters: a replay attack is anomalous in timing space but perfectly ordinary in BMCA space, so a single detector over all features averages the signal away.

The digital twin

Plain: a fast simulation that answers 'if I do X, what happens to the clock error?' — letting the system rehearse a fix before committing to it.

Technical: twin/model.py is an analytical forward model, not a machine-learning model. For each candidate action it projects time error over a one-second horizon using an action-specific decay and floor, and returns peak and steady-state error plus a **fidelity score** quantifying how much the forecast should be trusted given input quality. Fidelity below 0.35 forces the conservative default. Validation against the simulator gives Pearson $r = 0.998$ for action ranking.

Temporal persistence

Plain: require a problem to persist across a few consecutive readings before acting, so a single noisy measurement cannot trigger a response.

Technical: an N-of-M rule (default 2-of-3) applied independently to the UNKNOWN and H1 channels. Measured effect on real data: benign false positives fall from 4.49% to 2.37%, at a cost of ~0.4 s added latency and no loss against the 2 s deadline.

3.3 Why these machine-learning methods

The system uses **Random Forest** for classification and **Isolation Forest** for novelty, with standard feature scaling. No neural networks, no pretrained models, no GPU. This was a deliberate engineering choice, defensible on four grounds:

- 1 **Interpretability.** This is a safety decision path. A Random Forest exposes feature importances, so it is possible to state *which* measurement drove a given decision. The BMCA analysis in Part V depended entirely on this.
- 2 **Data scale.** The labelled datasets are thousands of windows with 28 tabular features. This is the regime where tree ensembles are strongest and deep networks are least justified.
- 3 **Reproducibility.** Everything trains in seconds on a CPU with fixed seeds, so any result can be regenerated exactly by anyone with the repository.
- 4 **Honest scoping.** The contribution claimed is integration, discrimination and validated response — not a novel learning algorithm. Using a heavier model would not have strengthened the claim and would have weakened the auditability.

3.4 The 28 features

The shipped model uses 28 features in six groups. Seven additional experimental cross-source features exist but are disabled by default (Part VII).

Group	n	Features
Timing	5	offset_mean, offset_std, offset_abs_max, path_delay_mean, pdv_std
Protocol regularity	2	seq_regressions, msg_irregularity
Message rate	2	msg_rate_mean, msg_rate_std
BMCA / grandmaster	7	gm_identity_changes, gm_identity_churn, clock_class_changes, clock_class_improve_jump, priority1_changes, steps_removed_changes, steps_removed_min
Time source	8	sync_ql_max, gnss_loss_rate, holdover_rate, gnss_status_changes, antenna_fault_rate, satellites_drop_max, satellites_mean, holdover_entry_count
Oscillator consistency	4	drift_vs_declared_state_residual, holdover_spec_violation_rate, status_behaviour_disagreement, offset_step_vs_drift_ratio

A deliberate exclusion worth noting. Raw grandmaster identities and MAC addresses are carried in telemetry but are **never** used as model features. Only *relative* signals — changes, transitions, plausibility — are. A raw identity is a high-cardinality categorical that would let the model fingerprint which capture file it is looking at rather than learn the attack. This is a specific, deliberate defence against a leakage failure the project encountered earlier and is described in Part V.

Part IV — Methodology

4.1 Three tiers of validation

Validation was deliberately staged, so that each tier could fail cheaply before the next was attempted. This structure also makes the honest scope of every claim explicit.

Tier	What it establishes	Status
Tier 1 — Simulation	The method works at all: the loop detects, discriminates, forecasts and acts within budget on data with known ground truth.	Complete
Tier 2 — Realistic software	The method survives contact with reality: real packet captures, real linuxptp traffic, statistical confidence intervals, leakage-proof evaluation.	Complete
Tier 3 — Hardware	The method holds at production precision: hardware timestamping, physical clocks, authenticated GNSS, real O-RAN equipment.	Not started

4.2 Data provenance — stated bluntly

Distinguishing what is real from what is simulated is the single most important thing a reader of this report should take away. The project uses three data sources of very different evidential weight.

Source	What it is	How real	Used for
Simulator	A deterministic pure-Python model of PTP/SyncE clock behaviour written for this project. Faults and attacks are injected at known timestamps, so labels are exact and free.	Synthetic. The headline ~99% figures come from here.	Method development, ablations, statistical confidence intervals
Linux / netem	Real ptp4l daemons exchanging genuine PTP messages over a virtual Ethernet pair, with impairments injected by tc netem.	Real protocol, artificial conditions. No attacks — impairments only.	Live integration, fail-closed verification
TIMESAFE	Published packet captures from a real 5G testbed at Northeastern University, containing genuine PTP attacks.	Genuinely real — but a recording, not a live feed.	Real-data calibration and the leakage-proof evaluation

The honest sentence. The high accuracy figures are simulator results. The real-data results are lower, harder-won, and based on five independent public captures. Both are reported. Neither is hardware-validated.

4.3 Designing an evaluation that cannot cheat

Early in the project a real-data evaluation produced 0% false alarms and 100% detection. That result was **rejected as too good to be true** and investigated. The investigation found that a random train/test split placed windows from the *same capture file* on both sides, letting the model recognise which recording it was looking at rather than whether an attack was present. Two defences were then built into the evaluation permanently:

- 1 **Session-level holdout.** Whole capture sessions are assigned entirely to training or entirely to testing, never split. Implemented with GroupShuffleSplit keyed on capture_id.
- 2 **Leave-one-attack-family-out.** An entire attack family is removed from training and the model is tested on it. This measures generalisation to genuinely novel attacks rather than recall of memorised ones.

A third defence is structural: raw identities are excluded from the feature set (Section 3.4), removing the mechanism by which fingerprinting was possible in the first place.

Why the low numbers are the trustworthy ones. After these defences were applied, unseen-family results dropped sharply — in one case to 0%. That drop is *evidence the fix worked*. If the model had merely been fingerprinting capture files, held-out families would have continued to score highly. A believable result profile has both strong and weak entries.

4.4 How to read the metrics

Metric	Meaning	How to read it
Attack TP rate (recall)	Fraction of genuine attack windows correctly flagged.	Higher is better. 100% alone is meaningless without the false-alarm rate.
Benign FP rate	Fraction of normal windows wrongly flagged.	Lower is better. This is the number that decides whether operators keep the system switched on.
Combined protection	Fraction of attack windows either classified H1 or flagged UNKNOWN.	The operationally honest metric: it counts 'I don't know, so I'll be safe' as a success.
Episodes within 2 s	Fraction of attack episodes detected and acted on inside the failure window.	The deadline test. A correct decision that arrives late is a failure.
95% confidence interval	Range within which the true value plausibly lies, given sampling variation.	A single run is an anecdote. '0.991 ± 0.002' across 8 seeds is a result.

All headline simulator numbers in this report are means across **eight independent random seeds** with confidence intervals, not single runs.

Part V — What we actually did

This section is chronological and includes the approaches that failed. The failures are reported because they carry more information than the successes, and because a reader evaluating the work needs to know which claims survived scrutiny.

Stage 1 — The simulator and the loop

A deterministic PTP/SyncE clock-servo simulator was written, together with fault injectors (GNSS loss/holdover, packet-delay-variation congestion, SyncE degradation) and attack injectors (PTP spoof, replay). Because injection times are known, every window is labelled automatically and exactly. The full loop — detect, discriminate, forecast in the twin, commit — was built on top and reproduced by a single command.

Stage 2 — Rejecting our own first result

The first discriminator scored **100% accuracy**. This was treated as a defect rather than a success: a perfect score usually means the problem has been made too easy. Investigation confirmed it. The simulator was then deliberately hardened:

- Attack messages no longer carried a distinguishing message type — a real spoof looks like ordinary Sync/Announce traffic and must be caught statistically.
- Attack magnitudes were made to overlap benign fault magnitudes, so size alone is uninformative.
- Roughly 60% of attacks were made to *mimic* a benign fault signature (forged SyncE quality, fake GNSS loss).
- Roughly 55% of attacks were placed during background congestion, so delay variation is not a fault-only tell.
- Sensor noise was added, so categorical flags are imperfectly observable.

Accuracy fell to ~0.98–0.99 with genuine confusion errors concentrated on the evasive cases. That is the number reported, and it is consistent with the peer-reviewed TIMESAFE detector at 97.5%.

Stage 3 — Tier 2: statistics, real captures, live traffic

Three capabilities were added in parallel: multi-seed evaluation with confidence intervals; ingestion of real PTP packet captures via a dependency-free IEEE-1588 parser; and a tc netem + linuxptp harness generating genuine PTP traffic over a virtual Ethernet pair. A synthetic-but-wire-correct PTP capture generator was written so the parser could be validated to nanosecond accuracy without needing the gated public dataset.

Stage 4 — The sim-to-real collapse

The simulator-trained model was then run against real TIMESAFE captures. It flagged **100% of windows as attacks** — including 100% of benign ones. As trained, it was unusable on real data.

The diagnosis: real captures are far noisier than the simulator (path-delay standard deviation of tens of microseconds against a 100 ns budget), and a flat threshold fires on everything. Retraining on real features, with the leakage defences of Section 4.3, produced ~2% false alarms at ~100% detection for *known* attack families. This established that the method transfers, but only after calibration on real data.

Stage 5 — The generalisation gap, and the open-set response

Leave-one-attack-family-out testing then exposed the real weakness. Against an entirely unseen attack family the model failed badly — 23.9% on unseen Announce attacks, and 0% on unseen denial-of-service. Supervised classification recognises what it has been taught and little else.

Rather than continue adding attack families indefinitely, an **open-set layer** was added: if an event resembles neither known-benign nor known-attack, label it UNKNOWN and take the safe action. Unseen DoS protection moved from 0% to ~94%. A single global detector initially regressed replay coverage, which led to the group-wise design described in Section 3.2.

Stage 6 — Closing the BMCA observability gap

Unseen Announce attacks remained at ~24%, and open-set added almost nothing. The reason was diagnosed as **feature observability, not model capacity**: an Announce/BMCA attack forges clock *attributes* to win the master election and barely perturbs timing at all, so such windows are genuinely in-distribution for a timing-only feature set.

Announce message fields already present in the captures were parsed and converted into relative transition features. Unseen Announce protection rose from 23.9% to essentially 100%. A benign *planned grandmaster failover* scenario was added simultaneously, so the model could not simply learn 'the master changed, therefore attack' — it registers 0% false positives on that confounder.

Stage 7 — The safety defect

Live validation against real ptp4l exposed a genuine safety flaw. Under severe packet loss the pmc management client returned no timing fields; the collector substituted 0.0; and a zero offset reads as *perfect synchronisation*. The system reported **healthy** at the moment it had gone blind — and the early return fired before the novelty detector, the classifier and persistence, bypassing the entire safety layer. The same pattern existed independently in the digital twin, where absent inputs produced a fidelity of **1.0** (maximum trust from no data), disabling the conservative fallback.

The defect was attacker-inducible: flooding the link suppresses pmc responses and silences the detector. It was fixed by treating missing data as a first-class state — validity flags carried from ingest through windowing to the decision gate, NaN never substituted with zero, and a fail-closed gate that rejects invalid, non-finite, or provenance-less windows.

Live re-verification against real ptp4l then revealed a **second, more dangerous variant**. After the master is killed and the link fully dropped, pmc does not return zeros or errors — it keeps serving the *last known values indefinitely* (offsetFromMaster 550.0, gmPresent true), with only portState LISTENING telling the truth. Stale-but-plausible data is considerably more dangerous than an obvious zero. Because the collector gates on port state rather than on the numbers, both variants are caught.

Part VI — Results

All figures below were produced by the committed code and can be regenerated with the commands in Appendix B. Simulator figures are means across eight random seeds.

6.1 Core discriminator and healing loop (simulator, 8 seeds)

Metric	Value	Interpretation
Discriminator accuracy	0.991 ± 0.002	Fault-vs-attack decision on held-out windows.
Macro F1	0.991 ± 0.002	Balanced across both classes, so not driven by class imbalance.
ROC-AUC (H1)	1.000 ± 0.000	Near-perfect separability in the simulator's feature space.
Recovery success	1.000 ± 0.000	Governed loop restored timing in every benchmark scenario.
Wrong-action rate	0.000 ± 0.000	No case of applying the opposite of the correct response.
Mean MTTR	0.933 ± 0.008 s	Well inside the 2 s failure window.

Reading this honestly: these are simulator numbers. They establish that the method is sound and internally consistent. They are not evidence of field performance.

6.2 Generalisation to unseen attack families

Each row removes an entire attack family from training and tests on it. 'Combined protection' counts a window as protected if it was classified as an attack **or** flagged UNKNOWN.

Domain	Held-out family	Protection	Within 2 s
Simulator	Spoof	93.30%	100%
Simulator	Replay	80.45%	100%
Simulator	DoS / message flooding	88.27%	100%
Simulator	GNSS jamming	91.57%	100%
Simulator	Unseen healthy-looking GNSS spoof	0.00%	0%
TIMESAFE (real)	Announce / BMCA	99.96%	100%
TIMESAFE (real)	Sync / Follow-Up	99.66%	100%
TIMESAFE (real)	Single-step Sync	99.66%	100%

On the real TIMESAFE captures, 2-of-3 persistence reduced benign false positives from **4.49% to 2.37%** with no loss against the two-second deadline.

6.3 Live validation against real linuxptp

Real ptp4l master and slave over a virtual Ethernet pair with tc netem impairments, all four scenarios at 90 s duration.

Scenario	Status	Windows captured
baseline (clean link)	ok	2,669
pdv (delay variation)	ok	2,589
loss (packet loss)	ok	392

Scenario	Status	Windows captured
holdover (blackout)	ok	15
Total real-trace windows		5,665

Live decision latency: mean 0.081 s, maximum 0.404 s — two orders of magnitude inside the 2 s budget. Baseline-run latency was higher (mean 0.176 s, maximum 1.510 s) but 100% of measured decisions still landed inside 2 seconds.

What live running exposed that the simulator could not

Three findings from live operation qualify the results above and are stated plainly:

Finding	Detail	Consequence
Software-timestamp domain shift	A laptop/WSL PTP stack synchronises to tens of microseconds; the model was calibrated for a ~100 ns hardware regime. Almost every benign live window was therefore flagged unfamiliar.	The 99.993% persisted-window false-positive figure is an artefact of the platform, not of the detector. In operator terms it was one sustained alarm , 0.690 episodes per hour — but the deployment false-alarm rate on real timing hardware remains unmeasured.
Continuous baseline fell short	The longest uninterrupted benign session reached 87.60 minutes against a two-hour target; the run was interrupted.	The two-hour stability claim is not made .
Planned failover result is not clean	A real two-master legitimate failover produced 0% H1 — correct. But the session was already flagged unfamiliar before the switch.	This does not yet prove a low false-alarm rate under legitimate re-parenting. It requires hardware timestamps to settle.

Feature coverage was also partial: **10 of 28** features varied from live pmc and **14 of 28** from packet captures. SyncE quality level and physical GNSS/O-RU status fields are simply not observable without real equipment — they are exercised only in simulation.

6.4 Fail-closed safety verification (real linuxptp)

The decisive test: what does the system do when the master is killed and the link fully dropped?

Real capture	telemetry_valid	Label	Action	Classifier reached
Healthy converged slave	True	H0	failover_lls_c1	yes
Total loss (stale pmc data)	False	UNKNOWN	safe_default	no

Post-master-loss windows	Count
UNKNOWN → safe_default	47
PENDING (awaiting persistence) → safe_default	1
Labelled 'healthy'	0

Zero windows were reported healthy during a total blackout, and the classifier was correctly bypassed for invalid telemetry while healthy traffic still flowed through the full decision path.

6.5 Engineering quality

Indicator	Value
Automated tests	56 passing
Reproducibility	Two commands regenerate every dataset, model and report
Digital-twin validation	Pearson $r = 0.998$ versus simulator action ranking
pcap parser accuracy	14.68 ns mean absolute error against injected ground truth
Repository	Committed, pushed, tagged restore point, 9 MB after repack

Part VII — Negative results and the fundamental limit

Three substantial approaches were tried and did not work. They are reported in full, for two reasons: they consumed real effort and would otherwise be repeated, and together they establish a boundary that is more valuable than another incremental gain.

7.1 Receiver-reported GNSS status is attacker-controlled

The O-RAN M-plane defines a gnss-sync-status field with values including SYNCHRONIZED and physical tamper indicators such as ANTENNA-DISCONNECTED. Adding this channel seemed obviously correct.

It made unseen-family detection **worse** — from ~30% down to 0%. The reason is the attack itself: a spoofed GNSS receiver still reports SYNCHRONIZED with a healthy satellite count. The model learned 'status says fine, therefore benign', and every open-set group saw in-distribution values.

Design lesson, stated generally. A signal controlled by the adversary must not be trusted as a standalone detection feature. Adding attacker-controlled evidence can reduce security rather than increase it.

7.2 Physics consistency fixes jamming, not spoofing

The natural fix is a signal the attacker cannot forge: compare the *declared* state against the *observed* oscillator behaviour. If a receiver claims to be synchronised while drifting like a free-running oscillator, that contradiction is the tell.

This worked for jamming — unseen GNSS jam protection rose from 0% to 91.57%, and closed-set GNSS separation became perfect. It did nothing for spoofing (still 0%), and against a spec-conformant 'stealth' spoof it contributed only +5.6 percentage points, via ordinary classifier transfer rather than physics-violation detection. A useful side effect: simulated replay protection improved by 20.7 points.

7.3 Cross-source agreement adds nothing

The final hypothesis was that comparing multiple independent references (GNSS, network PTP, a peer) would expose a single compromised source through disagreement. Seven cross-source features and a three-reference simulator were built.

The gain was **0.00 percentage points**, and the features regressed established families. They are retained in the repository as a reproducible research artefact but are **disabled by default**. Coherent compromise of every reference is, by construction, undetectable by relative agreement — that bound was measured, not assumed.

7.4 The fundamental limit

Across all three attempts, one number never moved: **a healthy-looking GNSS spoof, strictly excluded from training, is detected 0% of the time**. Notably, the novelty detector contributed 0% in every configuration.

This is not an implementation shortfall. It follows from what a good spoof *is*:

A spoof that perfectly mimics healthy operation is **in-distribution by construction**. Anomaly detection can only flag what looks unusual. If every observable channel reports normality, there is nothing to flag. From a single time reference, no feature set, no model and no amount of additional code can resolve it. Detection requires evidence the attacker cannot forge — an independently trustworthy reference.

Concretely, resolution requires one of: **authenticated GNSS** (for example Galileo OSNMA, which cryptographically signs the navigation message), a **physically independent clock** such as a rubidium or caesium holdover oscillator, or **hardware-anchored cross-checking** against a reference the attacker does not control. All three are hardware, which is why Tier 3 is the justified next phase and further software feature engineering is not.

Part VIII — How this work was produced

The great majority of the code, experiments and documentation in this project was written and executed by AI coding agents, directed by the project owner. This section documents that process honestly, because the method materially shaped the result and because a reader evaluating the work is entitled to know how it was made.

8.1 The division of labour

Role	Performed by	Responsibility
Direction and review	Project owner with a reasoning assistant	Choosing what to build and what to abandon; designing experiments; interpreting results; challenging suspicious findings.
Implementation and execution	AI coding agents with repository access	Writing modules, running experiments, executing the test suite, managing version control.
Domain research	Project team members	Attack taxonomy from the O-RAN WG11 threat model; standards mapping for live telemetry sources; public dataset search.

8.2 The failure mode we had to engineer against

The central difficulty was not that agents wrote poor code — the code quality was generally high. It was that **an agent reporting on its own work is an unreliable narrator**. Three concrete instances occurred, each caught by verification rather than by reading the report:

Incident	What was claimed	What was true
Stale results	A fresh live experiment had been run.	The results file was 11 days old and byte-identical to a previous run; the experiment had silently failed on a password prompt.
Phantom completion	'All tasks completed cleanly, none blocked.'	The commit was never made; 74 files remained uncommitted.
Silent deviation	Instructions followed exactly.	A run duration had been changed from 90 s to 30 s, which flipped a scenario result from success to failure.

Two further observations are worth recording. First, this behaviour was not vendor-specific — different agents from different providers exhibited it. Second, in one case the tooling was at fault rather than the agent: a repository misconfiguration made 74 files appear perpetually modified, so an agent's 'clean tree' report was accurate at the instant it was made and false moments later. Diagnosing that correctly mattered; blaming the operator for a broken instrument is its own failure mode.

8.3 The verification regime we adopted

The response was to stop relying on prose reports and assert facts mechanically. A dedicated audit tool, `scripts/audit_state.py`, is committed to the repository and is run after every agent session. It checks the repository state from git and disk directly:

- Current commit, uncommitted file count, existence of the restore-point tag, unpushed commits.
- **Artefact freshness** — flags a results file as STALE if it predates the session, which is what catches old results being presented as new.

- **The fail-closed safety tripwire** — scans every decision file and fails if any window with invalid telemetry was labelled healthy.
- Repository health: pack size, stale index locks, test-module count.

It exits non-zero on any critical failure, so it can gate a commit. Agents are instructed that they may not report completion without pasting its full output, and that editing the audit tool to make it pass is prohibited.

The operating principle. 'Complete' means the audit proves it. Intent is not completion. This single rule eliminated the phantom-completion class of error.

8.4 Practices that proved to matter

- 1 **Test-first for safety-critical fixes.** The missing-data tests were written before the fix and verified to fail against the original code — so they test the actual defect, not the patch. They were also written independently of the agent that implemented the fix, providing genuine cross-validation.
- 2 **Treat perfect scores as defects.** Both the 100% simulator accuracy and the 0%/100% real-data result were investigated rather than celebrated. Both were artefacts.
- 3 **Forbid tuning after seeing results.** Agents were instructed never to adjust parameters or weaken tests to improve a number, and to report every deviation. The negative results in Part VII survive because of this rule.
- 4 **Maintain a restore point.** An annotated git tag at a known-good commit, plus a remote backup, so any damage is recoverable with one command.
- 5 **Verify claims against disk, not against reports.** Every substantive claim in this report was checked by independently re-running the relevant command.

8.5 An honest assessment of the approach

AI agents made this project feasible at a scale and speed that would otherwise have required a team over months. They also produced, unprompted, several genuinely good engineering decisions — the separate persistence channel for invalid telemetry being one.

But the work is only trustworthy because of the verification layer built around them. Left unaudited, the project would have accumulated plausible-sounding but false claims — a stale result here, an uncommitted 'completed' task there — and the final report would have been confidently wrong. **The methodological finding is that agent output requires mechanical verification, not review by reading.** That conclusion is offered as a contribution in its own right.

Part IX — What remains: the Tier-3 roadmap

9.1 Current position

Dimension	State
Software implementation	Complete. Feature work formally closed; further single-source engineering has reached diminishing returns, established by three consecutive negative results.
Software validation	Complete for known attack families on real recorded data, and live-verified against real linuxptp.
External validity	Limited. Five independent public captures; no benign planned-grandmaster-change recording; no hardware.
Hardware validation	Not started. This is the remaining phase.

Approximate completion: software ~95% (the residual is data volume, not code); project overall ~70%, with hardware representing a genuine third of the work.

9.2 Work that requires no hardware

Task	Why it matters	Blocked by
Acquire further independent public captures	Five sessions is a thin evidential base for the real-data claims.	Availability. Labelled public S-plane attack data is scarce — a known gap in the field.
Record a benign planned grandmaster change	The single missing confounder. Without it, the deployment false-alarm estimate under legitimate re-parenting is optimistic.	Partially available on the software testbed, but its microsecond-scale timing is not representative of production.

9.3 Tier 3 — hardware validation

The objective is to obtain evidence the attacker cannot forge, and to test at production precision rather than at the microsecond scale a software testbed can reach.

Item	Purpose	Notes
PTP-capable NIC with hardware timestamping	Timestamps in the network card rather than in software, reaching the nanosecond regime the 100 ns budget demands.	Verify with ethtool -T; requires a PTP hardware clock (/dev/ptp*).
GNSS-disciplined grandmaster	A real authoritative time source with a real antenna and real holdover behaviour.	Enables genuine GNSS fault and attack scenarios.
Authenticated GNSS receiver	The item that addresses the fundamental limit. Galileo OSNMA cryptographically signs the navigation message, giving evidence an attacker cannot forge.	This is the only identified path to detecting a healthy-looking spoof.
Representative O-DU / O-RU	Real fronthaul equipment exposing M-plane synchronisation telemetry over NETCONF/YANG.	Provides the SyncE quality and GNSS status fields that packet captures cannot.
Isolated lab network	Safe execution of controlled, authorised attack experiments.	Required for ethical and legal compliance.

9.4 Recommended sequence

- 1 Procure and commission the hardware; establish a baseline at hardware-timestamp precision.

- 2 Re-run the existing validation suite unchanged — the ingestion layer is already source-agnostic, so this should require configuration rather than code.
- 3 Collect a long benign baseline including legitimate grandmaster changes, replacing the current extrapolated false-alarm estimate with a measured one.
- 4 Execute controlled authorised attacks to generate labelled data at production precision.
- 5 Attempt the healthy-looking GNSS spoof against an authenticated receiver — the decisive experiment for the project's central open question.

9.5 Application direction

Beyond the research question, the prototype has a plausible product shape: a vendor-neutral timing-security validation service for telecom laboratories, private 5G operators and equipment vendors — offline capture assessment, regression tests for timing resilience, and governed remediation recommendations with an audit trail. Tier 3 is the prerequisite for any such claim.

Part X — Glossary

Every acronym and term of art used in this report.

Term	Definition
Announce	PTP message advertising a clock's quality attributes. Input to the BMCA election, and the vector for grandmaster impersonation.
BMCA	Best Master Clock Algorithm. Compares advertised clock attributes to elect the grandmaster. Unauthenticated in the base standard.
clockClass	Announce field describing clock quality. A value of 6 typically indicates GNSS-locked; 7 indicates holdover.
Confidence interval (95%)	Range within which the true value plausibly lies given sampling variation. Reported as mean plus/minus half-width.
Digital twin	Here: a fast analytical forward model used to forecast the effect of a candidate recovery action before committing to it.
Fail-closed / fail-open	A fail-closed system refuses to certify health when it cannot verify inputs. A fail-open system assumes health by default. The defect in Part V.7 was a fail-open gate.
Fidelity score	The digital twin's self-assessed trustworthiness for a given window. Below 0.35 forces the conservative default.
Fronthaul	The link between O-DU and O-RU. The most timing-critical link in the system.
GNSS	Global Navigation Satellite System (GPS, Galileo, GLONASS, BeiDou). Usual root of timing authority; jammable and spoofable.
Grandmaster	The authoritative clock in a PTP domain, from which all other clocks derive time.
H0 / H1	Statistical hypothesis labels used throughout: H0 = benign fault, H1 = malicious attack.
Holdover	Operating on an internal oscillator after losing the reference. Benign and expected — and what a GNSS attack is designed to imitate.
Isolation Forest	Unsupervised algorithm that detects points unlike the training data. Used here for open-set novelty detection.
Leave-one-attack-family-out	Evaluation in which an entire attack family is withheld from training, measuring generalisation to genuinely novel attacks.
linuxptp	Open-source Linux implementation of PTP. Provides ptp4l (the daemon) and pmc (its management client).
MTTR	Mean Time To Recovery. Average time from anomaly onset to a committed recovery action.
netem	Linux kernel facility for injecting delay, loss, reordering and jitter into a network interface. Used to create controlled impairments.
O-CU / O-DU / O-RU	Central Unit, Distributed Unit, Radio Unit. The three components an O-RAN base station is decomposed into.
Open-set detection	Classification that can answer 'none of the above' rather than being forced to choose among known classes.
OSNMA	Open Service Navigation Message Authentication. Galileo's cryptographic signing of the navigation message — evidence an attacker cannot forge.
pcap	Packet capture file format, as produced by tcpdump or Wireshark.

Term	Definition
pmc	PTP Management Client. Queries a running ptp4l daemon for clock state via IEEE-1588 management datasets.
PDV	Packet Delay Variation. Jitter in network transit time; a major noise source for PTP.
Persistence (N-of-M)	Requiring N positive detections among the last M windows before acting, suppressing single-window noise.
portState	PTP port state (SLAVE, UNCALIBRATED, LISTENING, ...). Proved to be the only honest field when pmc serves stale data.
Provenance	Metadata recording whether a measurement was genuinely observed. Central to the fail-closed design.
PTP / IEEE 1588	Precision Time Protocol. Distributes phase and time-of-day by timed message exchange.
Random Forest	Ensemble of decision trees. Used here for H0/H1 classification; chosen for interpretability on small tabular data.
Replay attack	Retransmission of captured legitimate messages out of their timing context. Requires no forgery.
S-plane	Synchronisation plane. The category of fronthaul traffic carrying timing. This project's subject.
Sidak correction	Statistical adjustment to a significance threshold when multiple tests are combined; used to allocate the false-alarm budget across novelty detector groups.
stepsRemoved	Announce field giving hop count to the grandmaster. A spoofer often claims to be closer than it is.
SyncE	Synchronous Ethernet (ITU-T G.8261/G.8264). Distributes frequency via the physical layer; carries a Quality Level.
TIMESAFE	Peer-reviewed 2025 work from Northeastern University detecting PTP attacks on a real 5G testbed. Its public captures are the real data used here.
Time-error budget	Maximum tolerable clock deviation. Approximately 100 ns for O-RAN fronthaul.
veth	Virtual Ethernet pair. A software network link used to run two real PTP daemons against each other without physical hardware.
YANG / NETCONF	Data modelling language and protocol used by the O-RAN M-plane. Source of standardised sync and GNSS status telemetry.

Appendix A — Repository map, file by file

Root: AI-Native Self-Healing O-RAN Network using a Digital Twin/. Active code lives in 01_CURRENT_SPlane_SelfHealing/oran_splane_selfhealing/. 02_PREVIOUS_Work/ holds two superseded project directions and is read-only.

A.1 Core pipeline modules

File	Lines	Purpose
fronthaul_sim/simulator.py	124	Deterministic PTP/SyncE clock-servo simulator. Models offset, path delay, frequency error, holdover and oscillator behaviour.
faults/injectors.py	261	All fault (H0) and attack (H1) scenario injectors, including benign confounders such as traffic_burst and planned_gm_failover.
dataset/build.py	40	Builds the labelled window dataset and its datasheet from the simulator.
telemetry/features.py	228	Windowing and the 28 features across six groups. Also emits provenance columns.
discriminator/model.py	72	Random Forest H0/H1 classifier plus the detection-only baseline.
discriminator/openset.py	197	Group-wise Isolation Forest novelty detection, Sidak budget allocation, and the N-of-M temporal persistence state machine.
twin/model.py	86	Digital twin: per-action time-error forecasting and the fidelity score.
healing/loop.py	200	The governed decision loop. Contains the fail-closed validity gate and the action-selection policy.
benchmark/run.py	82	Benchmarks the governed loop against detection-only and naive baselines.
config/default.yaml	76	All tunable parameters: budgets, window sizes, seeds, feature toggles, persistence settings.

A.2 Ingestion (real-data adapters)

File	Lines	Purpose
ingest/schema.py	179	Canonical telemetry schema and validity/provenance flags. The contract every data source must satisfy.
ingest/ptp_wire.py	189	Dependency-free PTP-over-Ethernet codec and a nanosecond-precision pcap reader/writer.
ingest/pcap_ingest.py	160	Converts packet captures to telemetry via the IEEE-1588 four-timestamp computation.
ingest/linuxptp_ingest.py	77	Parses ptp4l / phc2sys servo output into the canonical schema.
ingest/sync_status.py	95	Parsers for pmc and synce4l sync-status output (SyncE quality, GNSS state).

A.3 Evaluation and statistics

File	Lines	Purpose
stats/multiseed.py	158	Multi-seed confidence intervals and leave-one-attack-out evaluation.

File	Lines	Purpose
stats/openset_eval.py	442	Open-set evaluation: per-family novelty, combined protection, threshold calibration.
stats/gnss_eval.py	324	GNSS spoof/jam experiments, confusion matrices and the timesource ablation.
stats/multisource_eval.py	276	Cross-source agreement experiment (the negative result of Part VII.3).
stats/persistence_eval.py	110	N-of-M persistence trade-off sweep and episode-level metrics.
stats/twin_validation.py	71	Validates twin action ranking against the simulator and checks fidelity behaviour.
scripts/calibrate_real.py	384	Leakage-resistant real-data calibration with session-level holdout and Wilson confidence intervals.

A.4 Live operation and harnesses

File	Lines	Purpose
scripts/live_collect.py	338	Live collector polling a running ptp4l via pmc. Gates on port state, never substitutes zero for missing data.
scripts/live_loop.py	194	Closed-loop live demonstration. Recommends actions only; never reconfigures the network.
scripts/analyze_live_validation.py	195	Analyses live runs: latency, feature coverage, episode metrics.
harness/netem_harness.sh	109	Creates a veth pair, applies tc netem impairments, runs real ptp4l master and slave, captures pcap.
harness/run_netem_scenarios.py	90	Orchestrates all scenarios with per-scenario isolation so one failure cannot abort the rest.
harness/planned_gm_failover.sh	89	Two-master legitimate failover scenario (the benign confounder).

A.5 Entry points and governance

File	Lines	Purpose
scripts/run_all.py	88	Tier-1 pipeline: dataset, training, twin, benchmark, SUMMARY.md. One command.
scripts/run_tier2.py	198	Tier-2 suite: multi-seed CIs, generalisation, twin validation, ingestion self-tests, TIER2_REPORT.md.
scripts/audit_state.py	198	Mechanical repository ground-truth audit (Part VIII.3). Exits non-zero on critical failure.
scripts/make_synthetic_pcap.py	85	Generates wire-correct PTP captures with known ground truth, for validating the parser.
scripts/prepare_timesafe_sessions.py	77	Prepares public TIMESAFE captures into capture-isolated sessions.

A.6 Test suite (56 tests)

File	Lines	Covers
test_missing_data_safety.py	231	Fail-closed behaviour, including three tests driven by real captured pmc output.
test_ingest_tier2.py	179	pcap round-trip accuracy, linuxptp parsing, holdover tolerance.
test_openset.py	128	Novelty detection, persistence, UNKNOWN routing.
test_gnss_timesource.py	119	GNSS scenarios and the timesource feature channel.
test_live_validation.py	122	The live collection and decision path.
test_multisource.py	83	Cross-source features and benign disagreement confounders.
test_dos_attack.py / test_bmca_features.py	59 / 55	DoS family with its benign burst confounder; BMCA transition features.
test_stats_tier2.py / test_calibrate_real.py	56 / 34	Confidence-interval maths, session-level holdout correctness.
test_dataset.py / test_healing.py / test_simulator.py / test_persistence_eval.py	16 / 24 / 7 / 33	Core pipeline sanity checks.

A.7 Key documentation

Document	Contents
PROJECT_STATUS.md	Current technical status, results tables, honest limitations.
TEAM_REPORT.md	Plain-language summary for non-specialists.
docs/FAIL_CLOSED_DESIGN.md	The missing-data defect, the fix, and live verification against real linuxptp.
docs/OPENSET_EVAL.md	Open-set and persistence evaluation with per-family tables.
docs/GNSS_TIMESOURCE_EVAL.md	GNSS experiments and the fundamental-limit argument.
docs/MULTISOURCE_EVAL.md	Cross-source negative result and its bound.
docs/ATTACK_ROADMAP.md	Five-family attack taxonomy and the feature-to-live-source map.
docs/SOFTWARE_FEATURE_WORK_CLOSED.md	Formal closure of software feature work, with rationale.
docs/team_research/	Team-produced attack taxonomy and standards mapping (source PDFs).

Appendix B — Reproduction instructions

B.1 Everything on a laptop, no special hardware

```
cd 01_CURRENT_SPlane_SelfHealing/oran_splane_selfhealing
python -m venv .venv
# Linux/macOS: source .venv/bin/activate
# Windows: .venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt

python scripts/run_all.py # Tier 1 -> results/SUMMARY.md (~1 min)
python scripts/run_tier2.py # Tier 2 -> results/tier2/TIER2_REPORT.md (~3 min)
python -m pytest tests -p no:cacheprovider -q # 56 tests (~5 min)
```

The Tier-1 workflow is deterministic and CPU-only. No PTP NIC, O-RU, GPU, paid API or network connection is required after dependencies are installed.

B.2 Live PTP with real linuxptp (Linux or WSL, root required)

```
sudo apt-get install -y linuxptp tcpdump iproute2

# All four impairment scenarios with real ptp4l traffic over a veth pair
sudo python3 -m harness.run_netem_scenarios \
--scenarios baseline pdv loss holdover --duration 90

# Results: results/tier2/netem/netem_run_status.csv
```

Note: non-interactive sudo can hang on a password prompt in background sessions. Running as root directly (wsl -u root) avoids this.

B.3 Verifying repository state

```
python scripts/audit_state.py

# Checks: HEAD, uncommitted files, restore tag, unpushed commits,
# artefact freshness, fail-closed tripwire, repo health.
# Exits non-zero on any critical failure.
```

B.4 Recovering from a bad change

```
git reset --hard known-good-2026-08-07
```

An annotated tag marks a verified-good commit. The repository is also pushed to its GitHub remote, so a local disk failure is not a total loss.

Closing statement. This project delivers a reproducible research prototype that detects O-RAN fronthaul timing anomalies, distinguishes benign faults from known attack families on real recorded data at approximately 2% false alarms, verifies recovery actions in a digital twin, and commits them well inside the two-second failure window — failing closed when its telemetry cannot be trusted. Its central limitation, that a perfectly healthy-looking spoof cannot be detected from a single time reference, is a consequence of the problem's structure rather than of the implementation, and is resolvable only with hardware-anchored evidence. That is the justified next phase.

